Large Language Modeling

Setup and submission. Use PyTorch tensors, layers, autograd, and AdamW, but no high-level Transformer implementation or downloaded model weights. Use the supplied `ByteTokenizer` unchanged ($V = 261$); tokenizer training is not assigned. Submit your derivations, runnable code, and the tables/plots requested below. CPU is sufficient; one CUDA GPU or MPS device is optional. Record the device and seed in your notebook. Poor generated text does not lose points if the implementation and analysis are correct. See `homework/llm/README_data.md` for data sources, splits, and preprocessing.

1. Causal multi-head attention

Let $X \in \mathbb{R}^{B\times T\times d}$ with $B = 2$, $T = 5$, $d = 12$, and $H = 3$ attention heads. Each head has width $d_h = d/H$. Use no biases or dropout in parts (a)–(b).

(a) **Equations and shapes.** Write the query, key, and value projections, scaled score matrix, additive causal mask, row-wise softmax, head concatenation, and final output projection. Give the shape after every operation, including the head axis. State exactly which score entries must be $-\infty$, and explain why the mask is applied *before* softmax.

(b) **Two equivalent implementations.** Implement one attention head using explicit matrix multiplication and another using three `nn.Linear` layers. Copy the *same* projection weights between them, accounting for PyTorch's weight-storage convention. On a seeded random X, report the maximum absolute output difference and assert agreement with `atol=rtol=1e-6`.

(c) **A test of causality.** Using the complete two-block GPT specified in Problem 2, create two length-5 integer sequences that agree at zero-based positions $0, 1, 2$ and differ at $3, 4$. In evaluation mode, assert that their logits at positions $0, 1, 2$ agree to the same tolerance. Report the measured maximum difference.

2. Decoder architecture and parameter accounting

Use the local `GPTConfig` conventions:

$$V = 261, \quad C = 256, \quad L = 2, \quad H = 2, \quad d = 64, \quad d_{\mathrm{FF}} = 4d.$$

Here C is maximum context length and L is the number of decoder blocks. All attention and feed-forward linear layers have biases. Each LayerNorm has learned scale and shift. The output vocabulary projection has no bias and shares its weight with the token embedding. Use dropout probability zero for Problems 2–4.

The decoder has token and learned position embeddings; L pre-normalized blocks; a final LayerNorm; and a vocabulary projection. Each block has two LayerNorms, multi-head attention with Q, K, V and output projections, and a feed-forward network $d \to 4d \to d$ with GELU.

For the training problems, you may use the supplied `math550.topics.llm.GPT` after checking its structure against your derivation. You need not rewrite a second training framework.

(a) **Residual block.** Write the two residual-update equations for a pre-normalized block. Implement the block using your attention computation and ordinary PyTorch layers. Verify that an input of shape $(2, 16, 64)$ preserves its shape and that a scalar loss produces finite gradients.

(b) **Count, then verify.** Derive symbolic parameter counts for token embeddings, position embeddings, attention, the feed-forward network, the two block LayerNorms, and the final LayerNorm. Give the total for this configuration and check it against `sum(p.numel() for p in model.parameters())`. How the attention count changes if only the three Q, K, V biases are removed while the output-projection bias is retained.

(c) **Mode versus gradients.** Distinguish `model.eval()` from `torch.no_grad()`. Demonstrate them with a standalone `nn.Dropout(p=0.5)` layer and an input requiring gradients: show a training-mode output, an evaluation-mode output, and whether each output tracks gradients. This diagnostic does not change the GPT's zero-dropout configuration.

3. Next-token pretraining and controlled decoding

Use the cleaned text of H. G. Wells's *The Time Machine* (Project Gutenberg eBook 35). Split at $\lfloor 0.9 \operatorname{len}(\text{text}) \rfloor$ Python characters *before* encoding: the prefix is training data and the suffix is validation data. Encode each split separately with the fixed byte tokenizer and append one EOS. Never draw a window across the split.

(a) **Objective.** Derive the autoregressive chain-rule factorization and the mean next-token negative log-likelihood for input $x_{1:T}$ and target $x_{2:T+1}$.

(b) **One pretraining run.** Set seed 550 *before* creating the Problem 2 model. Train for exactly 100 optimizer updates with batch size 4, window length 256, and AdamW: learning rate 10^{-3}, weight decay 0.01, and $\beta = (0.9, 0.999)$. Use no schedule, mixed precision, or gradient clipping. Sample window starts uniformly with replacement using a separate CPU `torch.Generator` seeded 551.

At updates $0, 25, 50, 75, 100$, evaluate the *same* first 16 nonoverlapping length-256 input/target windows of each split (starts $0, 256, \ldots, 3840$). Use evaluation mode, no gradients, and total negative log-likelihood divided by scored targets. Plot both loss curves; report final validation perplexity and the number of training target positions processed. Save the update-100 model for Problem 4. Give a short explanation of the observed curves.

(c) **Temperature in isolation.** For logits $z = (0, 1, 2)$ and $\tau \in \{0.5, 1, 2\}$, calculate $p^{(\tau)} = \operatorname{softmax}(z/\tau)$. With seed 550 reset for each temperature, draw 10,000 categorical samples and compare empirical frequencies with these probabilities. Report the maximum absolute discrepancy. How does the temperature affect concentration?

(d) **Decode the checkpoint.** From the prompt "The Time Traveller", generate at most 64 new tokens with top-$k = 40$, separately at temperatures 0.7 and 1.2; reset the sampling seed to 550 for each run and stop at EOS. Also generate with an explicit greedy argmax. Display all three continuations. Explain why the same seed can still give different sampled continuations at different temperatures, why different seeds need not give different text, and when top-$k = 1$ is equivalent to greedy decoding. Use evaluation mode for every continuation.

4. Instruction following with a precisely defined SFT task

Task. Instruction following means producing a response that performs the requested operation on the supplied input and satisfies the stated constraints. Here the tasks include grammatical correction, unit conversion, and sentence rewriting.

Run `prepare_instruction_data.py` in this homework directory. It uses the pinned book repository's `instruction-data.json` and maps the source fields `instruction`, `input`, and `output` to `InstructionExample` fields using explicit keywords (`instruction=`, `context=`, `response=`), and records all source indices. The test rows are the eight zero-based indices in the table on the next page. Of the remaining distinct prompts, the first 40 eligible rows in source order form 32 training and 8 validation examples. Eligibility requires at most 192 prompt tokens and at most 257 prompt-plus-response tokens, including the final EOS. No example is truncated.

Use the supplied `format_instruction_prompt` without modification; it includes system/user/assistant control tokens. Encode the stripped reference response separately and append one EOS. Freeze the tokenizer.

(a) **Specify and test labels.** Let a concatenate p prompt tokens and r response tokens, including EOS. With Python's half-open slice convention, form inputs $a[0 : m-1]$ and targets $a[1 : m]$, where $m = p + r$, then right-pad to length 256. Derive the exact zero-based target indices that receive loss under response-only training; all other labels are -100.

Work out both label arrays (response-only and full-sequence, excluding padding) for the toy sequence $a = (10, 11, 12, 13, 20, 21, 256)$ with $p = 4$, $r = 3$, and padded input length 8. Here 256 is EOS; 257 is the input padding ID. For one real training row, print the prompt length, first response target index, EOS target index, and number of scored targets. Assert these values, prompt exclusion, EOS inclusion, and padding exclusion.

(b) **Define the two objectives.** Write full-sequence and response-only mean cross-entropy, with their respective denominators.

(c) **Controlled fine-tuning.** Make two independent copies of the update-100 checkpoint from Problem 3. Fine-tune one with full-sequence labels and one with response-only labels. Use fresh AdamW optimizers with Problem 3's settings, batch size 4, and exactly 100 updates per copy. Precompute one sequence of training-example indices, sampled uniformly with replacement using a CPU generator seeded 552, and reuse it in both runs. Only the loss mask may differ.

Report response-only token-weighted negative log-likelihood on all training and validation examples before and after fine-tuning. At the end, also report it on all eight test examples for the base checkpoint and both fine-tuned models.

(d) **Generate and score actual answers.** For every test row, give each of the three models only the formatted prompt (instruction and optional input, *not* the reference answer). Use greedy argmax, evaluation mode, a maximum of 64 new tokens, and stop at the first EOS. Retain the original prompt and append only generated tokens.

Exclude the prompt and stopping EOS from each answer. Save all 24 continuations. For each, assign a binary task-success score using the row-specific criteria below; a response must meet every criterion in its row to score 1. Briefly justify each score. Also report a separate strict reference-match rate after case folding and collapsing whitespace, without removing punctuation. Report the two rates, each out of 8, for each model. Explain one case where these two scoring rules could disagree, and whether the lower held-out response negative log-likelihood corresponds to better task success in your run.